

Računarska vežba 4

PYTHON FUNKCIJE

1 Deklarisanje i pozivanje funkcije, prenos parametara

Funkcije omogućavaju:

- višestruku upotrebu dela programskog koda i time skraćanje dužine programa
- bolju organizaciju kôda programa njegovom podelom na manje celine, koje je lakše razumeti

Korisnička funkcija se definiše posebnom naredbom oblika:

```
def <nazivFunkcije>(<lista_parametara>):  
    <blok naredbi>
```

Definicija funkcije sastoji se od **zaglavlja** koje čine naziv funkcije i lista parametara i **tela** funkcije, koje čine naredbe grupisane radi izvršavanja nekog zajedničkog zadatka.

Promenljive u listi parametara koje preuzimaju aktuelne vrednosti iz poziva funkcije su **parametri**, a same vrednosti koje se prenose funkciji prilikom njene upotrebe su **argumenti** funkcije. Izvršavanje funkcije završava poslednjom naredbom ili naredbom **return**, a rad programa se nastavlja od prve sledeće naredbe iza poziva funkcije.

Korisnički definisana funkcija može da vrati neku vrednost kao rezultat pomoću posebne naredbe u telu funkcije, koja ima oblik **return <vrednost>**. Dozvoljeno je da funkcija istovremeno vrati i više vrednosti naredbom oblika **return <vrednost1>, <vrednost2>, ...**. Funkcije ne moraju da vrate nikakvu vrednost, odnosno mogu da završe samo naredbom **return** ili čak ibez nje i tada se vraća posebna vrednost **None**.

Funkcija se u programu koristi u izrazima na isti način kao i ugrađene funkcije, pomoću naziva i liste aktuelnih argumenata funkcije.

1.1 Deklarisanje funkcije bez parametara (bez return iskaza):

Funkcija može biti deklarirana bez parametara i bez navođenja `return` povratne vrednosti.

Primer:

```
1  # Primer: Deklaracija funkcije BEZ PARAMETARA
2
3  def pozdrav():
4      print("Welcome to Python !!!!")
5
6  # poziv funkcije navođenjem njenog imena
7  pozdrav()
8
```

Welcome to Python !!!!

Primer:

```
1  # Primer: Deklaracija funkcije BEZ PARAMETARA
2
3  def saberi_2_broja ():
4      broj_1 = 3
5      broj_2 = 6
6      zbir = broj_1 + broj_2
7      print(zbir)
8  saberi_2_broja() # poziv funkcije
9
9
```

Primer:

```
1  # Primer 3: Deklaracija funkcije BEZ parametara
2
3  def generisi_full_name ():
4      ime = 'Petar'
5      prezime = 'Petrović'
6      space = ' '
7      full_name = ime + space + prezime
8      print(full_name)
9  generisi_full_name () # poziv funkcije
10
```

Petar Petrović

1.2 Deklarisanje funkcije bez parametara (sa return iskazom):

Funkcija može vraćati vrednosti, iako nema ako naredbu `return`. U tom slučaju vrednost funkcije je `None`.

Primer:

```
1 # Primer: Funkcija bez parametara i sa return
2
3 def saberi_2_broja ():
4     br_1 = 5000
5     br_2 = 4500
6     zbir = br_1 + br_2
7     return zbir
8 print(saberi_2_broja())
9
```

9500

Primer:

```
1 # Primer: Funkcija bez parametara i return
2
3 def generisi_full_name ():
4     ime = 'Petar'
5     prezime = 'Petrovic'
6     space = ' '
7     full_name = ime + space + prezime
8     return full_name
9 print(generisi_full_name())
10
```

Petar Petrovic

1.3 Deklarisanje funkcije sa parametrima:

Funkciji se mogu proslediti različiti tipovi parametara (broj, string, logička vrednost, lista, torka, rečnik ili skup).

Funkcije sa jednim parametrom:

Primer:

```
1 # Primer: Funkcija sa jednim parametrom
2
3 def pozdrav(ime):
4     """
5     This function greets to the person passed in as a parameter
6     """
7     print("Hello, " + ime + ". Pozdrav!")    # Nema izlaz!
8
```

Primer:

```
1  # Primer: Funkcija sa jednim parametrom
2
3  def zbir_brojeva(n):
4      zbir = 0
5      for i in range(n+1):
6          zbir+=i
7      print(zbir)
8  print(zbir_brojeva(100)) # 5050 za prvih 100
9  print(zbir_brojeva(1000)) # 500500 za prvih 1000
10
```

5050

None

500500

None

Funkcije sa dva parametra:

Primer:

```
1  # Primer: Funkcija sa dva parametra
2
3  def predmet(ime, naziv_predmeta):
4      print("Pozdrav", ime, "izučavate Python")
5      print("Naziv predmeta je", naziv_predmeta)
6
7  predmet('Petre', 'Strukture podataka i algoritmi') # poziv funkcije
8
```

Pozdrav Petre izučavate Python

Naziv predmeta je Strukture podataka i algoritmi

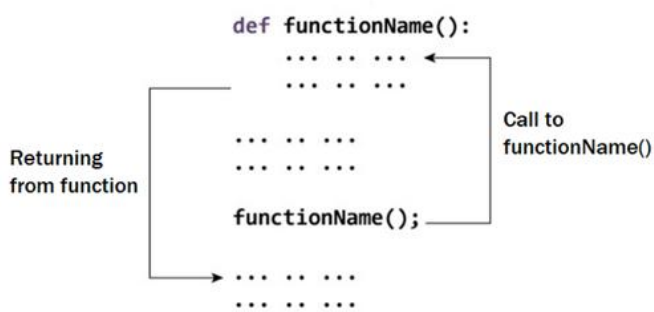
Primer:

```
1  # Primer funkcije
2
3  def kalkulator(a, b): # dva parametra
4      saberi = a + b
5      return saberi # vraća zbir
6
7  rezultat = kalkulator(300, 150) # poziv funkcije
8      # uzima se return vrednost promenljive
9  print("Zbir je:", rezultat)
10
```

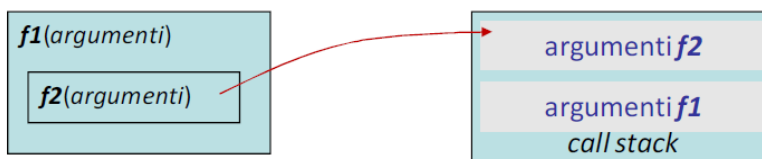
Zbir je: 450

1.4 Pozivanje funkcije:

Funkcija se poziva prema svom nazivu, uz navođenje liste aktuelnih argumenata. U jeziku Python se iz funkcija mogu pozivati druge funkcije, tako da su dozvoljeni ungnježdeni pozivi. Takođe je dozvoljeno da funkcija sadrži poziv same sebe, odnosno dozvoljeni su i rekurzivni pozivi.



Prilikom svakog aktiviranja funkcije, koja kao programski kôd postoji u jednom primerku, različiti aktuelni parametri funkcije iz svakog poziva čuvaju se u posebnoj sistenskoj strukturi, tzv. steku izvršavanja ili steku poziva (*call stack*). Argumenti se koriste obrnutim redom od redosleda uključivanja u strukturu stek po principu LIFO (Last In-First Out). Funkcija se izvršava za vrednosti argumenata koje se nalaze na vrhu steka (*top*), tj. prvo se izvrši za argumente funkcije *f2*, a nakon toga funkcije *f1*.



Primer:

```

1  # Primer: Poziv funkcije
2
3  def pozdrav(ime):
4      """
5      Ovom funkcijom pozdravlja se osoba čije se ime prosleđuje kao parametar
6      """
7      print("Pozdrav, " + ime + ". Da uspešno savladaš Python!")
8
9  pozdrav('Petar')
10

```

Pozdrav, Petar. Da uspešno savladaš Python!

Primer:

```

1  # Primer: Poziv funkcije
2
3  def generisi_full_name (ime = 'Milan', prezime = 'Milic'):
4      space = ' '
5      full_name = ime + space + prezime
6      return full_name
7  print(generisi_full_name())
8  print(generisi_full_name('Ana', 'Pavlović'))
9  |

```

Milan Milic
Ana Pavlović

Primer:

```

1  # Primer funkcije
2
3  def tezina_objekta (masa, grav_const = 9.81):
4      tezina = str(masa * grav_const)+ ' N' # vrednost promenjena u string
5      return tezina
6  print('Tezina objekta u Njutnima na Zemlji: ', tezina_objekta(100)) # Težina na površini Zemlje
7  print('Težina objekta u Njutnima na Mesecu: ', tezina_objekta(100, 1.62)) # Težina na površini Meseca
8  |

```

Tezina objekta u Njutnima na Zemlji: 981.0 N
Težina objekta u Njutnima na Mesecu: 162.0 N

1.5 Prenos parametara po vrednosti:

Prilikom izvršavanja proceduralnih programa, prenos argumenata funkciji može se vršiti **po vrednosti** ili **po referenci**. Prenos po vrednosti znači da se funkciji prenosi trenutna vrednost promenljive ili rezultata računanja nekog izraza, dok prenos po referenci znači da se funkciji prenosi samo referenca na vrednost, odnosno informacija o njenom položaju. Kad je aktuelni argument promenljiva, prenos po referenci omogućava funkciji da pristupi promenljivoj i promeni njenu vrednost, dok prenos po vrednosti funkciji prenosi samo kopiju trenutne vrednosti promenljive, koja je smaštena u sistemski stek poziva.

Prenos argumenata funkciji u jeziku Python vrši se **po vrednosti** (*pass-by-value*). Navođenje naziva promenljive kao aktuelnog parametra samo kopira vrednost promenljive, koja se smešta u stek i zatim koristi u telu funkcije. Ako argumente prosleđujemo sa ključem i vrednošću, redosled argumenata nije važan.

Primer:

Funkcija `povecaj` neće promeniti vrednost promenljive `brojac`, čija vrednost se prenosi kao argument, već samo lokalnu vrednost parametra `x`:

<code>def povecaj(x):</code>		
<code>x = x + 1</code>	<code>x</code>	00000002
<code>brojac = 1</code>	<code>brojac</code>	00000001
<code>povecaj(brojac)</code>	<code>brojac</code>	00000001

Primer:

```

1  # Primer prenosa parametara funkcije
2
3  def zbir(a,b): # Funkcija 1
4      print("Sabraci dve vrednosti")
5      print("Printing unutar funkcije")
6      print(a+b)
7      return a+b
8
9  def poruka(): # Funkcija 2
10     print("Učimo fukcije...Srećno")
11     return
12
13  ukupno=zbir(100,50)
14  print('Zbir dva broja je: ',ukupno)
15  poruka()
16  print("Rest of code")
17

```

```

Sabraci dve vrednosti
Printing unutar funkcije
150
Zbir dva broja je: 150
Učimo fukcije...Srećno
Rest of code

```

2 Lambda funkcije

Lambda funkcije u Python programiranju predstavljaju anonimne funkcije ili funkcije bez imena. Lambda je kratka i ograničena funkcija koja se sastoji od jednog reda kôda. Kao i regularna funkcija, lambda funkcija može imati više argumenata s jednim izrazom.

Svaka anonimna funkcija u Python jeziku ima 3 dela:

- ključna reč `lambda`
- parametri ili argumenti (vezane promenljive)
- izraz

`lambda argumenti: izraz`

Lambda funkcija sadrži samo jedan izraz čiji rezultat vraća. Ona ne poseduje naredbu return i ne sadrži naredbe ili višestruke izraze. Lambda funkcije koriste sopstveni prostor imena. Argumenti se prosleđuju lambda funkciji. Izraz je bilo koji važeći Python izraz sa parametrima koje smo dali funkciji.

Primer:

```
1  saberi = lambda x, y: x + y
2  print (saberu (50, 250))
3
```

300

U ovom primeru se definiše promenljiva koja će da sadrži rezultat koji vraća lambda funkcija. Ključna reč lambda se koristi za definisanje anonimne funkcije, dok su x i y parametri koje se prosleđuju lambda funkciji. U telu funkcije sabiraju se dva parametra koja smo prosledili i to je jedan izraz. U okviru print pozivamo funkciju i ispisujemo vraćenu vrednost.

Primer:

```
1  # Primer: Lambda funkcija, sortiranje liste
2
3  result=sorted([7,3,1,4,2,22,15], key=lambda x:x)
4  print(result)
```

[1, 2, 3, 4, 7, 15, 22]

Primer:

Funkcija `map()` primenjuje (mapira) lambda funkciju na svaki element iterativne liste. U ovom primeru, lambda funkcija kvadrira svaki element liste.

```
1  # Primer: Lambda funkcija, primer sa map()
2
3  brojevi=[5,10,50,100]
4  kvadrati=map(lambda x:x**2, brojevi)
5  print(list(kvadrati))
```

[25, 100, 2500, 10000]

Primer:

Funkcija `filter()` filtrira svaki element iterativne listen a osnovu datog uslova. U ovom primeru, lambda funkcija prikazuje samo parne brojeve liste.

```
1 # Primer: Lambda funkcija, primer sa filter()
2
3 brojevi=[1,2,3,4,5,6,7,8,9,10]
4 parni_brojevi=filter(lambda x:x%2==0, brojevi)
5 print(list(parni_brojevi))
```

```
[2, 4, 6, 8, 10]
```

Primer:

Funkcija `sorted()` koristi se za sortiranje podataka. U ovom primeru, lambda funkcija „kaže“ `sorted()` da koristi drugi element (poene) za sortiranje elemenata torke.

```
1 # Primer: Lambda funkcija, primer sa sorted()
2
3 studenti=[("Ana",85),("Petar",65),("Dragan",90),("Milica",55)]
4 # Sortiranje prema poenima
5 stud_sort=sorted(studenti, key=lambda x:x[1])
6 print(stud_sort)
```

```
[('Milica', 55), ('Petar', 65), ('Ana', 85), ('Dragan', 90)]
```

Primer:

Lambda funkcije mogu se kombinovati sa standardnim funkcijama.

```
1 # Primer: Kombinovanje lambda i standardne funkcije
2
3 def mnozac(n):
4     return lambda a: a*n
5 dvostruko=mnozac(2)
6 trostruko=mnozac(3)
7 print(dvostruko(55), trostruko(111))
```

```
110 333
```

Osnovna razlika između lambda i standardne funkcije je u tome što lambda funkcija koristi samo jedan izraz i daje funkcijski objekt. Zbog toga možemo imenovati rezultat lambda funkcije i koristiti ga u program. Kod standardnih funkcija moramo definisati njeno ime, i ona vraća rezultat kada se pozove.

Lambda funkcija ne sadrži naredbu return jer ima samo jedan izraz koji se uvek vraća prema zadatim uslovima.

Zadaci za vežbu

1. Napisati funkciju pod nazivom `add_all_nums` koja prihvata proizvoljan broj argumenata i sabira sve argumente. Proverite da li su svi elementi liste brojevnog tipa. Ako nisu, ispisati odgovarajuću povratnu poruku.

```
1 # Primer: Funkcija sabiranje brojeva i provera ....
2
3 def add_all_nums(*nums):
4     for i in nums:
5         if type(i) != int and type(i) != float:
6             return f"ERROR: Svi argumenti moraju biti brojevi! {i}: {type(i)}"
7     return sum(nums)
```

2. Napisati funkciju pod nazivom `godisnje_doba`, koja kao parameter uzima mesec i vraća naziv godišnjeg doba: jesen, zima, proleće ili leto. Zadatak pokušati rešiti preko strukture rečnika.

Primer rešenja!

```
1
2 # Primer: Funkcija meseci i godisnja doba ....
3 def godisnje_doba(mesec):
4     mesec_gdoba = {
5         "Januar": "Zima",
6         "Februar": "Zima",
7         "Mart": "Prolece",
8         "April": "Prolece",
9         "Maj": "Prolece",
10        "Jun": "Leto",
11        "Jul": "Leto",
12        "Avgust": "Leto",
13        "Septembar": "Jesen",
14        "Oktobar": "Jesen",
15        "Novembar": "Jesen",
16        "Decembar": "Zima"
17    }
18    if type(mesec) != str or mesec.capitalize() not in mesec_gdoba:
19        return f"Argument mora biti mesec: {mesec} ({type(mesec)}"
20    else:
21        return mesec_gdoba.get(mesec.capitalize())
```

3. Kvadratna jednačina je oblika $ax^2 + bx + c = 0$. Napisati funkciju `solve_quadratic_eqn` koja izračunava skup rešenja kvadratne jednačine, a rešenje se dobija preko izraza

$$x = (-b \pm \sqrt{b^2 - 4ac})/2a$$

4. Napisati funkciju pod nazivom `reverse_list` koja uzima niz kao parametar i vraća obrnutu vrednost niza (koristiti petlju FOR).

```
print(reverse_list([1, 2, 3, 4, 5]))
# [5, 4, 3, 2, 1]
print(reverse_list(["A", "B", "C"]))
# ["C", "B", "A"]
```

5. Napisati funkciju pod nazivom `sum_of_odds` koja kao parameter uzima broj `n` i sabira sve neparne brojeve u tom opsegu do tog broja.

```
def sum_of_odds(n):
    return sum(num for num in range(n + 1) if num%2 != 0)
```

6. Napisati funkciju pod nazivom `parni_neparni` koja uzima pozitivan ceo broj `n` kao parametar i prebrojava parne i neparne brojeve u opsegu do `n`. Napisati poziv funkcije.

```
1  # Primer: Broj parnih i neparnih ....
2
3  def parni_neparni(n):
4      parni, neparni = 0, 0
5      for i in range(n+1):
6          if i % 2 == 0:
7              parni += 1
8          else:
9              neparni += 1
10     return f"Broj parnih je: {parni}\nBroj neparnih je: {neparni}"
11
12 print(parni_neparni(100))
```

```
Broj parnih je: 51
Broj neparnih je: 50
```

7. Napišite funkciju koja proverava da li su svi elementi liste isti tip podataka. Implementirati poziv funkcije za različite liste.

```
1  # Primer: Tip elemenata liste ....
2
3  def is_type(lst):
4      _type = set()
5      for i in lst:
6          _type.add(type(i))
7      if len(_type) == 1:
8          return f"{True} - Svi elementi liste su istog tipa ({type(lst[0])})"
9      else:
10     return f"{False} - Postoji {len(_type)} razlicitih tipova u listi ({_type})"
```

8. Napisati program implementacijom funkcije kojom se zamenjuju vrednosti dve promenljive. Koristiti privremenu memorijsku lokaciju temp.

```
1  # Primer: Zamena mesta brojeva
2
3  def swap(x, y):
4      """
5      Funkcija zamenjuje vrednosti dve promenljive
6      """
7      temp = x; # vrednost x ide u temp
8      x = y;    # vrednost y ide u x
9      y = temp; # vrednost temp ide u y
10     print("Vrednost za x je:", x)
11     print("Vrednost za y je:", y)
12     return    # "return" je opciono
13
14     x = 66
15     y = 55
16     swap(x, y)    #poziv funkcije
17
```

Vrednost za x je: 55

Vrednost za y je: 66

9. Napisati funkciju za sabiranje dva broja koja će vraćati 4 vrednosti u return iskazu, svaka vrednost da bude po jedna matematička operacija (sabiranje, oduzimanje, množenje i deljenje)

```
1  # Primer prenosa parametara funkcije
2
3  def aritmet_oper(broj1, broj2):
4      saberi = broj1 + broj2
5      oduzmi = broj1 - broj2
6      pomnozi = broj1 * broj2
7      podeli = broj1 / broj2
8      # return 4 vrednosti
9      return saberi, oduzmi, pomnozi, podeli
10
11     a, b, c, d = aritmet_oper(10, 7) # procitaj 4 return vrednosti
12
13     print("Zbir je: ", a)
14     print("Razlika je: ", b)
15     print("Proizvod je: ", c)
16     print("Količnik je: ", d)
17
```

Zbir je: 17

Razlika je: 3

Proizvod je: 70

Količnik je: 1.4285714285714286

Funkcije

- To su delovi koda koji se mogu koristiti po potrebi preko „poziva funkcija“
- Funkcije se ne izvršavaju u program **dok se ne „pozovu“**
- Karakteristike funkcije:
- Ima **ime**
- Ima **parametre** (0 ili više njih)
- Ima specifikaciju ili **docstring** (opciono ali se preporučuje)
- Ima telo ili **blok naredbi**
- funkcija obično „nešto vraća“ preko **return**

Kako se piše i poziva funkcija

keyword ime parametri ili
argumenti

```
def is_even( i ):
```

```
    """
```

```
    Input: i, a positive int
```

```
    Returns True if i is even, otherwise False
```

```
    """
```

```
    print("inside is_even")
```

```
    return i%2 == 0
```

specifikacija,
docstring

blok komandi
funkcije

```
is_even(3)
```

u kodu se može pozvati funkcija
navođenjem njenog imena i
vrednosti parametara

Blok naredbi funkcije

```
def is_even( i ):  
    """  
    Input: i, a positive int  
    Returns True if i is even, otherwise False  
    """
```

```
print("inside is_even")
```

izvršavanje
naredbi

```
return i%2 == 0
```

izraz se izračunava i vraća se
vrednost

keyword

Opseg promenljivih

- **formalni parametar** se vezuje za vrednost stvarnog parametra kada se funkcija pozove
- novi **opseg/okvir/okruženje** se formira u okviru funkcije
- **opseg (scope)** je mapiranje imena u objekte

```
def f(x): Formalni parametar  
    x = x + 1  
    print('in f(x): x =', x)  
    return x
```



Definicija funkcije

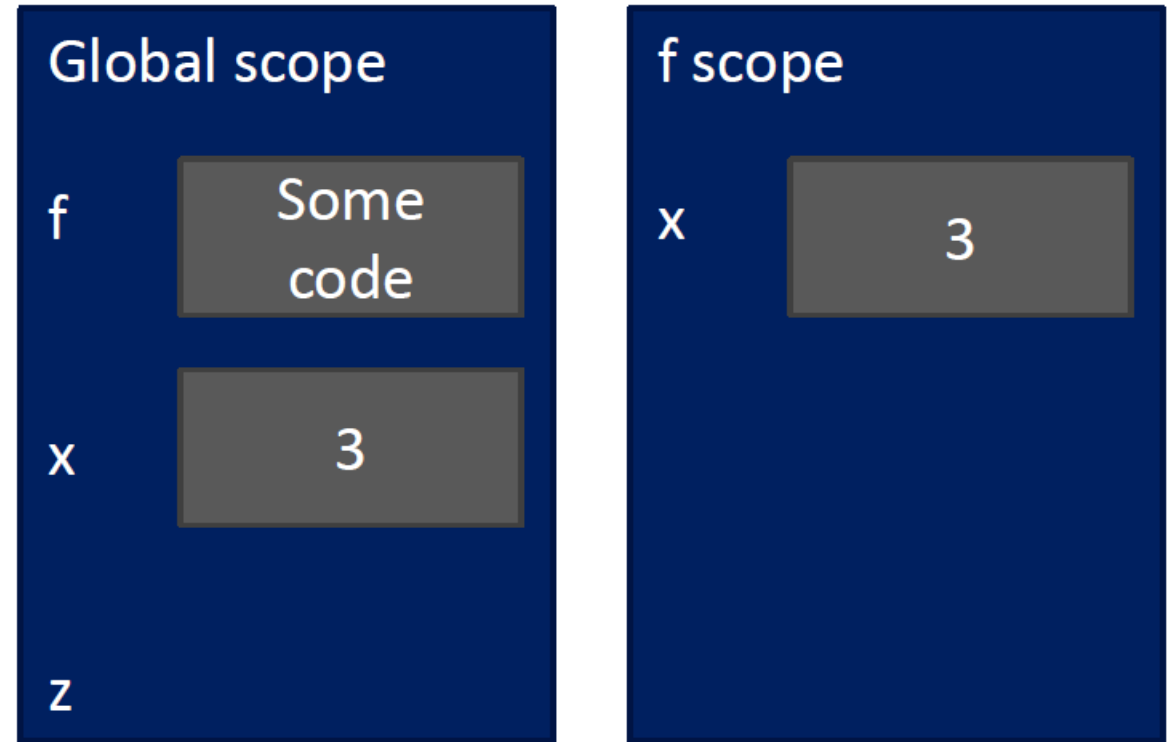
```
x = 3  
z = f(x) Stvarni parametar
```



Glavni programski kod:
* inicijalizuje promenljivu **x**
* poziva funkciju
* dodeljuje povratnu vrednost promenljivoj **z**

Opseg promenljivih

```
def f( x ) :  
    x = x + 1  
    print('in f(x): x =', x)  
    return x  
  
x = 3  
z = f( x )
```

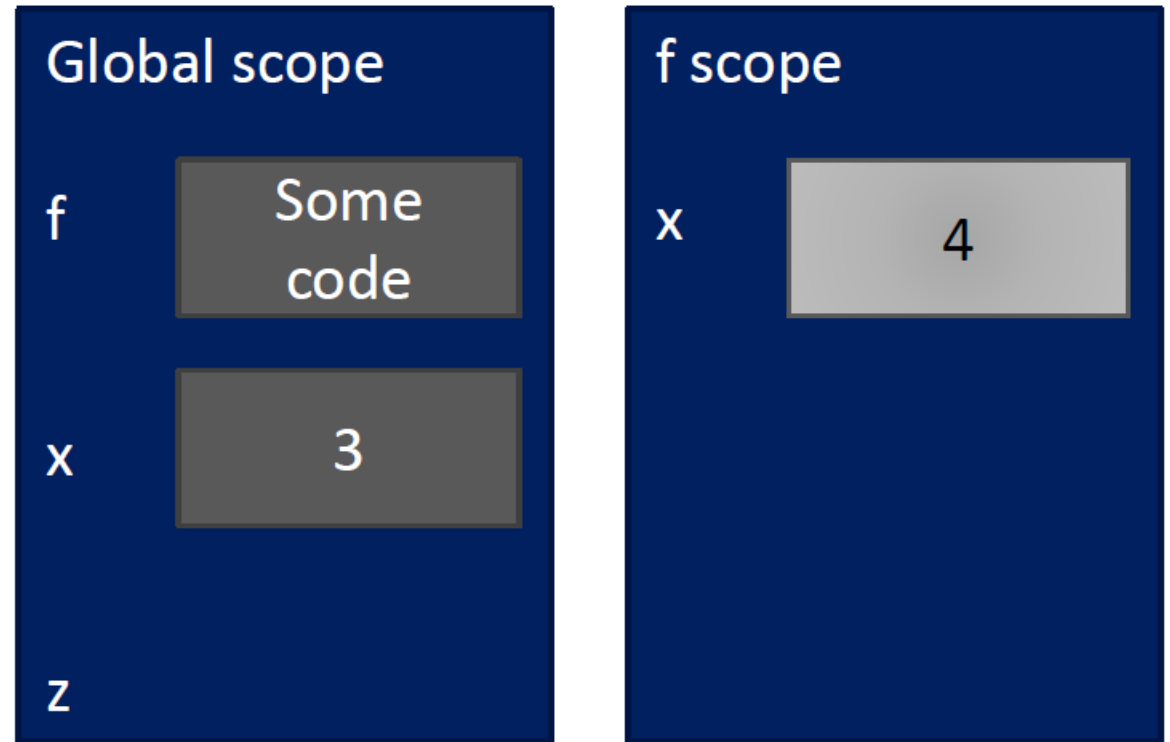


Opseg promenljivih

```
def f( x ) :  
    x = x + 1  
    print('in f(x): x =', x)  
    return x
```

```
x = 3
```

```
z = f( x )
```

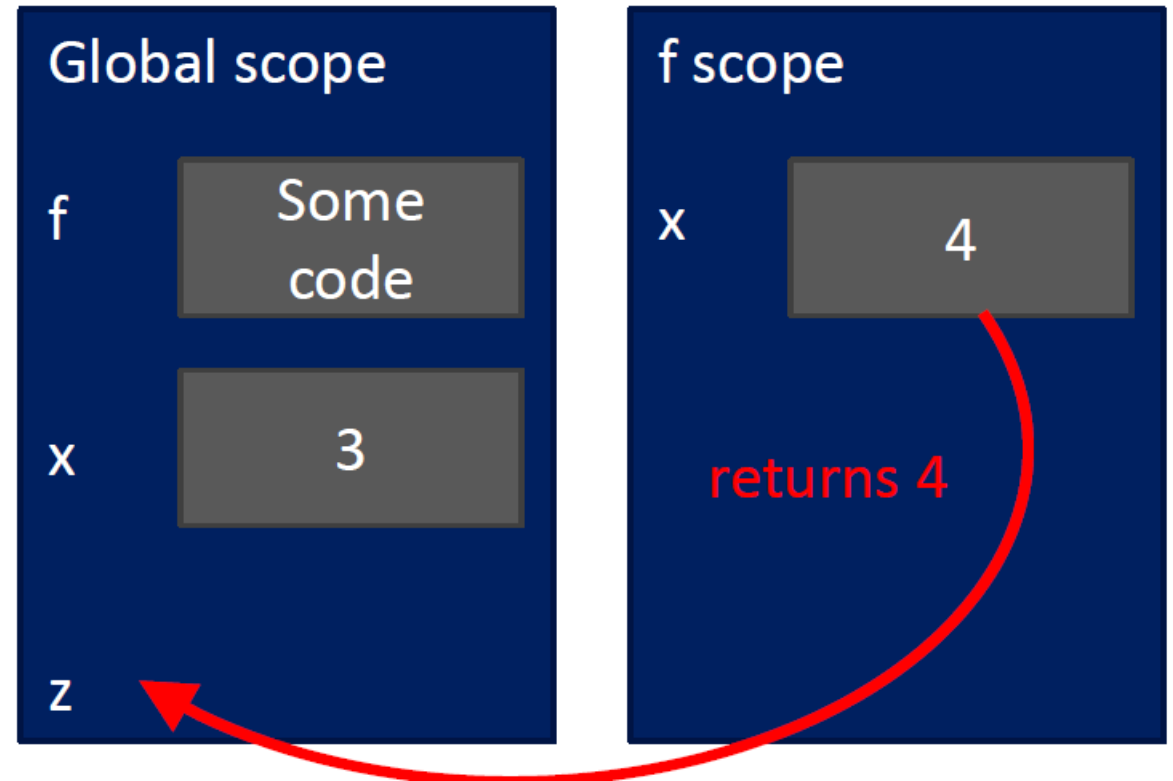


Opseg promenljivih

```
def f( x ) :  
    x = x + 1  
    print('in f(x) : x =', x)  
    return x
```

```
x = 3
```

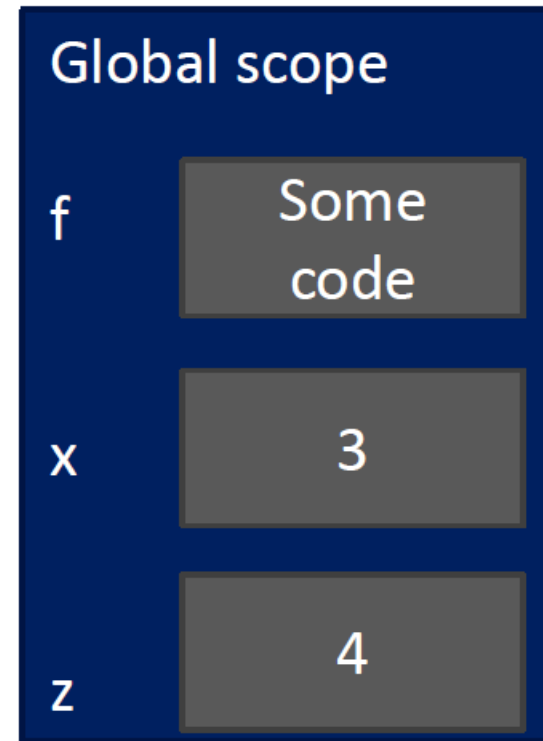
```
z = f( x )
```



Opseg promenljivih

```
def f( x ) :  
    x = x + 1  
    print('in f(x): x =', x)  
    return x
```

```
x = 3  
z = f( x )
```



Funkcija bez `return` iskaza!!!

- Ako nije navedena ključna reč `return`, program vraća vrednost **None**
- **None** označava odsustvo vrednosti

```
def is_even( i ) :  
    """  
    Input: i, a positive int  
    Does not return anything  
    """
```

```
i%2 == 0
```

bez RETURN
iskaza !!!

return vs. print

- **return** ima značenje samo **unutar** funkcije
- samo jedan **return** se može izvršiti unutar funkcije
- kôd koji je unutar funkcije ne izvršava se nakon naredba **return**
- ima neku pridruženu vrednost

- naredba **print** može se koristiti van funkcije
- može se više **print** naredbi izvršiti unutar funkcije
- kôd koji je unutar funkcije može se izvršiti i nakon naredbe **print**
- ima neku pridruženu vrednost, koja se prikazuje u promptu kao izlaz

Funkcije kao argumenti

- Argumenti mogu biti bilo kojeg tipa, a i funkcije mogu biti argumenti

```
def func_a():  
    print 'inside func_a'  
  
def func_b(y):  
    print 'inside func_b'  
    return y  
  
def func_c(z):  
    print 'inside func_c'  
    return z()
```

```
print func_a()
```

poziv **func_a**, ne uzima parametre

```
print 5 + func_b(2)
```

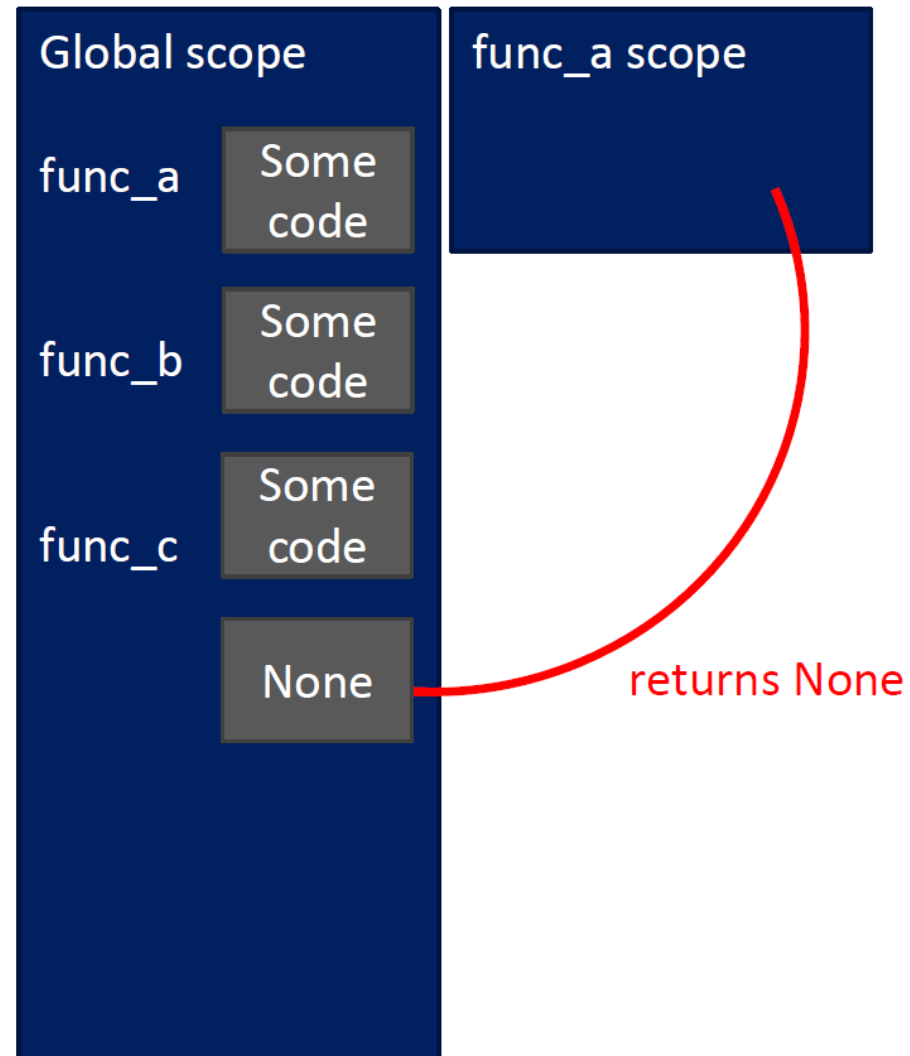
poziv **func_b**, uzima jedan parametar

```
print func_c(func_a)
```

poziv **func_c**, uzima jedan parametar i
to funkciju **func_a**

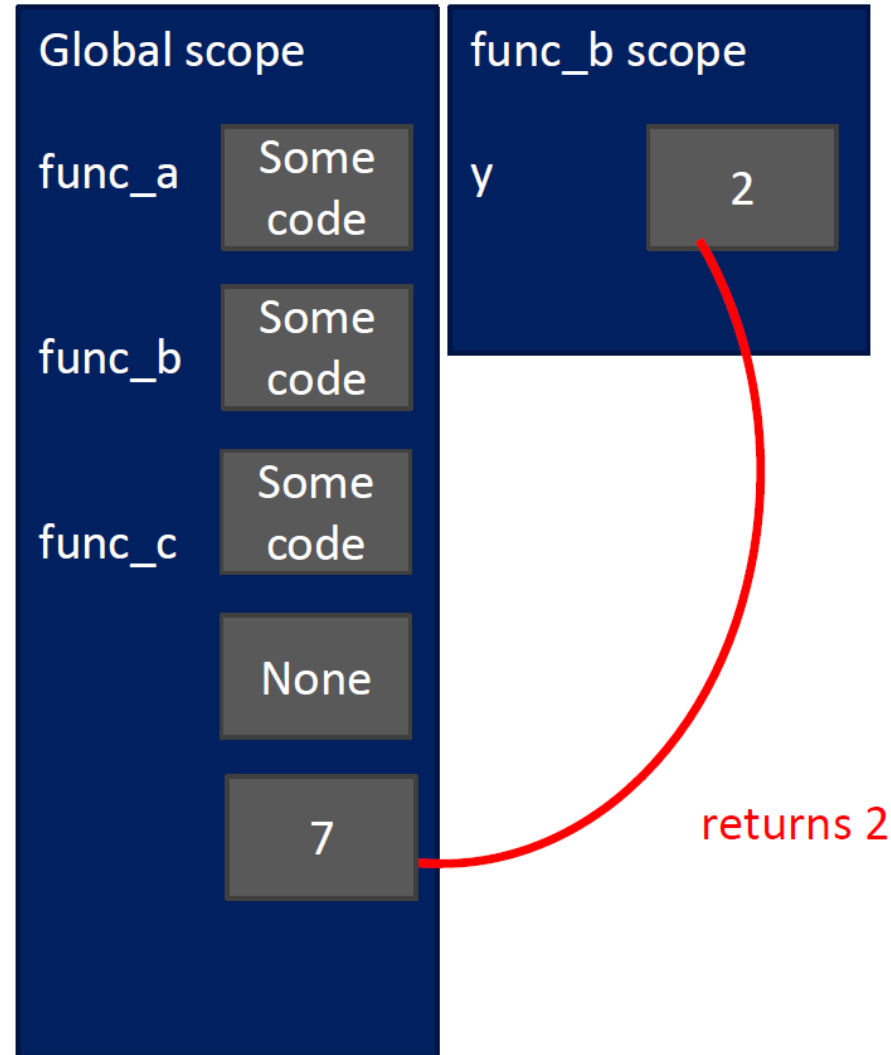
Funkcije kao argumenti

```
def func_a():  
    print 'inside func_a'  
def func_b(y):  
    print 'inside func_b'  
    return y  
def func_c(z):  
    print 'inside func_c'  
    return z()  
  
print func_a()  
print 5 + func_b(2)  
print func_c(func_a)
```



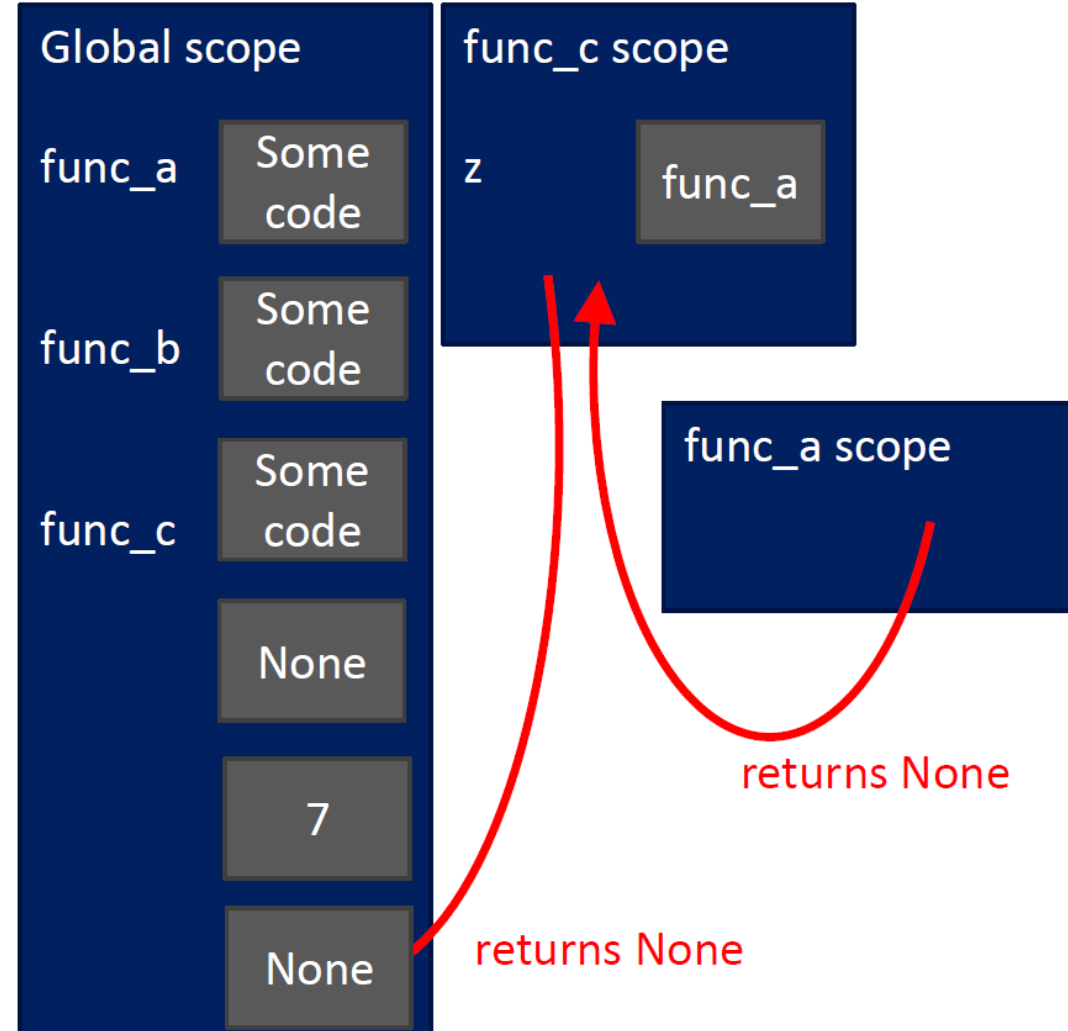
Funkcije kao argumenti

```
def func_a():  
    print 'inside func_a'  
def func_b(y):  
    print 'inside func_b'  
    return y  
def func_c(z):  
    print 'inside func_c'  
    return z()  
print func_a()  
print 5 + func_b(2)  
print func_c(func_a)
```



Funkcije kao argumenti

```
def func_a():  
    print 'inside func_a'  
def func_b(y):  
    print 'inside func_b'  
    return y  
def func_c(z):  
    print 'inside func_c'  
    return z()  
print func_a()  
print 5 + func_b(2)  
print func_c(func_a)
```



Funkcije kao argumenti

- Unutar funkcije, **može se pristupiti promenljivoj** definisanoj van nje
- Unutar funkcije, **ne može se menjati promenljiva** definisana van nje - mogu se koristiti globalne promenljive

```
def f(y):  
    x = 1  
    x += 1  
    print(x)  
  
x = 5  
f(x)  
print(x)
```

x je redefinisana u opsegu f

različiti x objekti

```
def g(y):  
    print(x)  
    print(x + 1)  
  
x = 5  
g(x)  
print(x)
```

x van g

x unutar g uzima se iz opsega koji je pozvao funkciju g

```
def h(y):  
    x += 1  
  
x = 5  
h(x)  
print(x)
```

UnboundLocalError:
lokalna promenljiva x je referencirana pre dodele vrednosti

Funkcije kao argumenti

- Unutar funkcije, **može se pristupiti promenljivoj** definisanoj van nje
- Unutar funkcije, **ne može se menjati promenljiva** definisana van nje - mogu se koristiti globalne promenljive

```
def f(y):  
    x = 1  
    x += 1  
    print(x)
```

```
x = 5  
f(x)  
print(x)
```

```
def g(y):  
    print(x)
```

```
x = 5  
g(x)  
print(x)
```

```
def h(y):  
    x += 1
```

```
x = 5  
h(x)  
print(x)
```

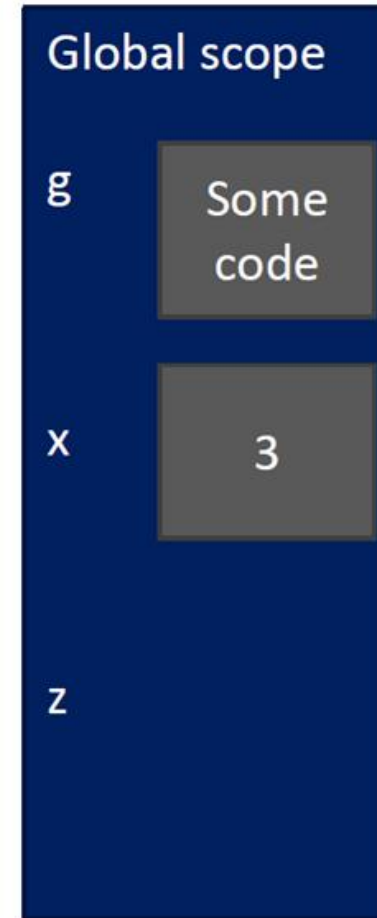
**x iz global/main
opsega programa**

Opseg - primer

```
def g(x):  
    def h():  
        x = 'abc'  
    x = x + 1  
    print('g: x =', x)  
    h()  
    return x
```

Programski
kod

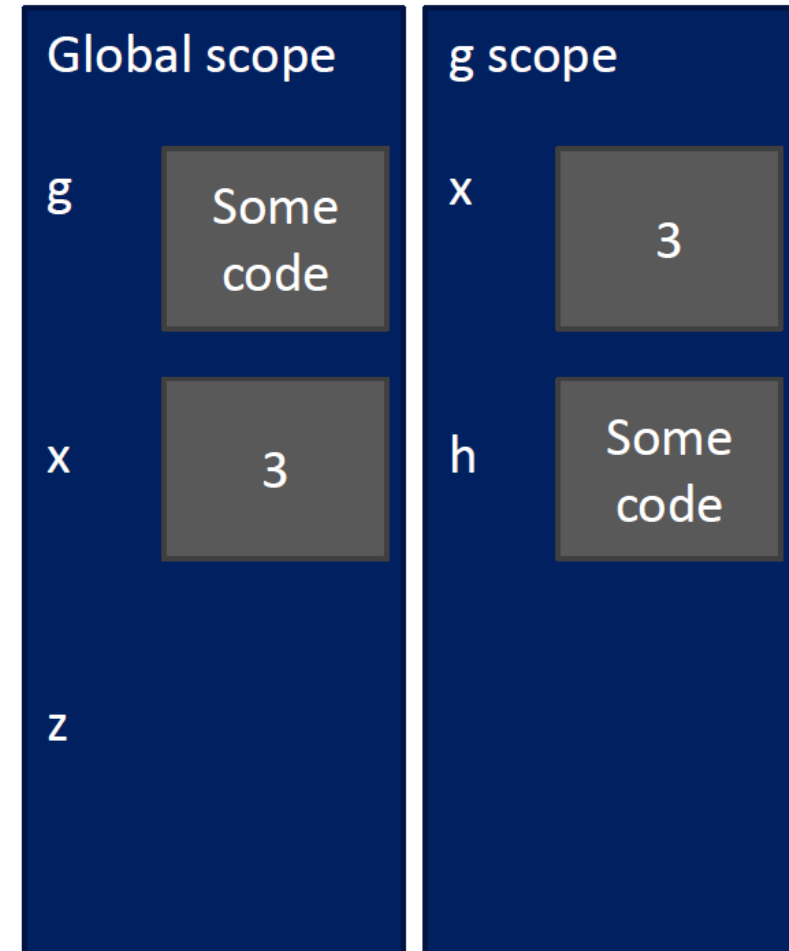
```
x = 3  
z = g(x)
```



Opseg - primer

```
def g(x):  
    def h():  
        x = 'abc'  
    x = x + 1  
    print('g: x =', x)  
    h()  
    return x
```

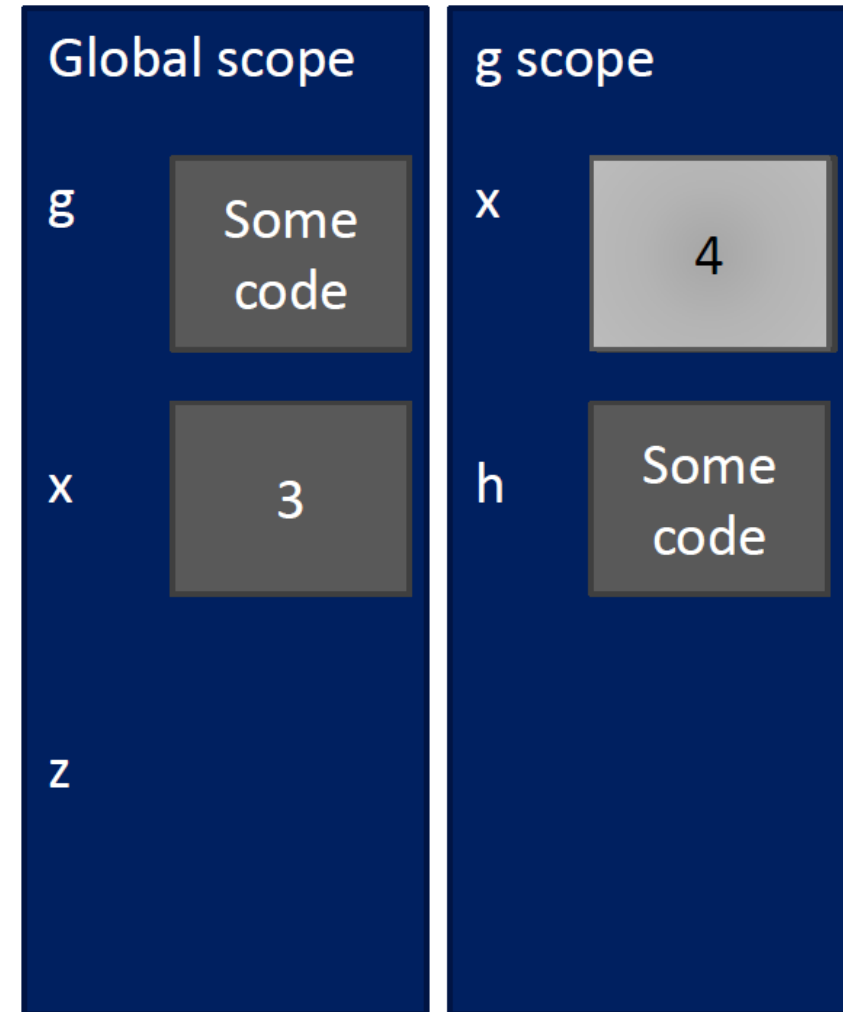
```
x = 3  
z = g(x)
```



Opseg - primer

```
def g(x):  
    def h():  
        x = 'abc'  
    x = x + 1  
    print('g: x =', x)  
    h()  
    return x
```

```
x = 3  
z = g(x)
```

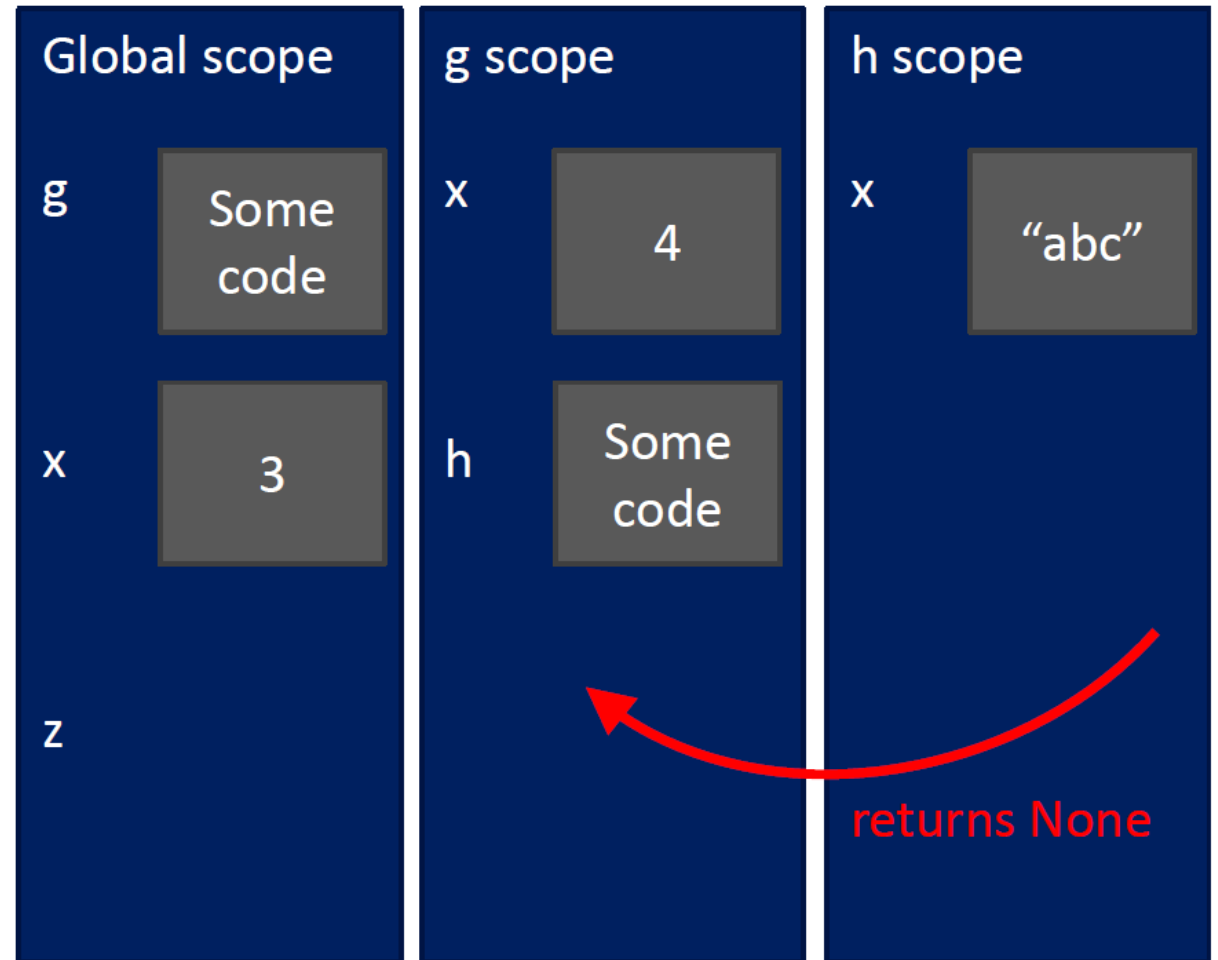


Opseg - primer

```
def g(x):  
    def h():  
        x = 'abc'  
    x = x + 1  
    print('g: x =', x)  
    h()  
    return x
```

```
x = 3
```

```
z = g(x)
```

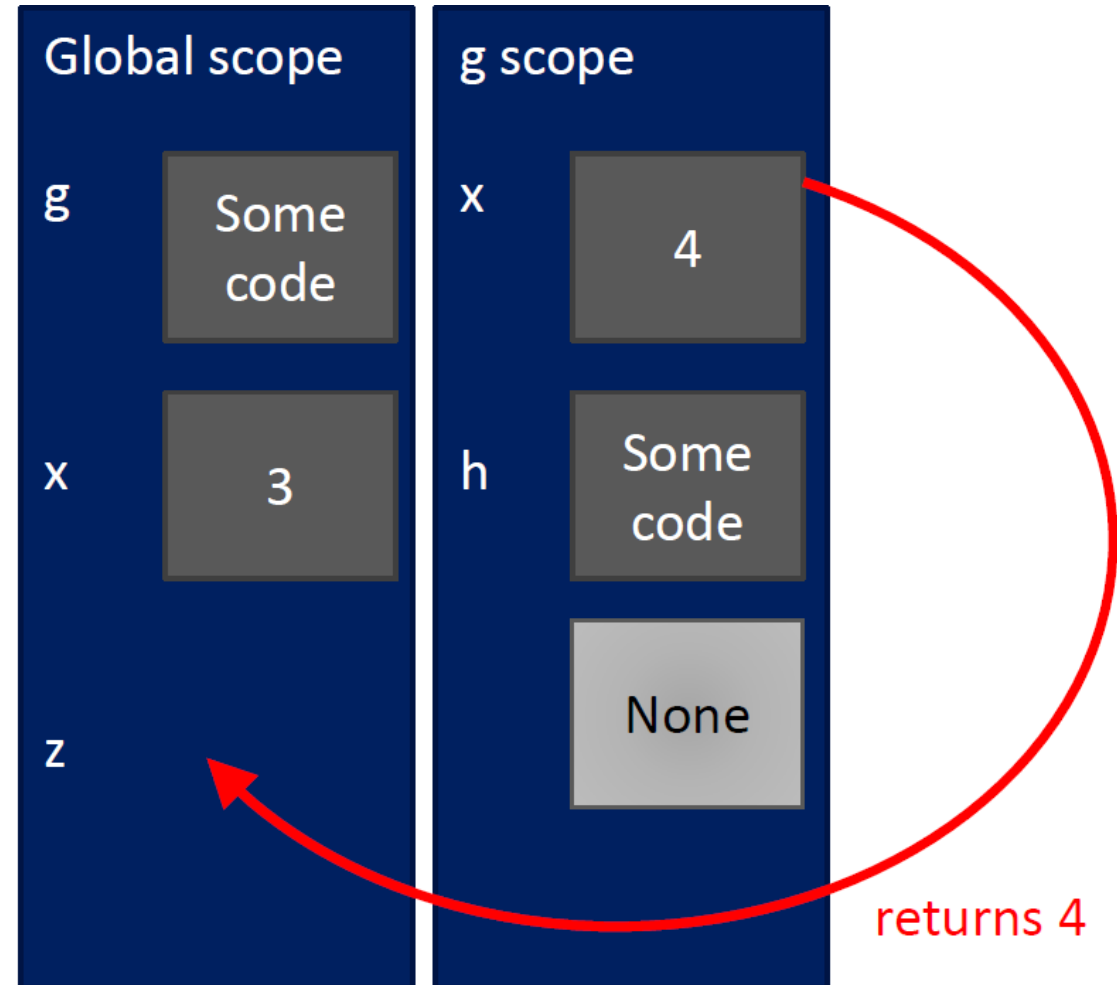


Opseg - primer

```
def g(x):  
    def h():  
        x = 'abc'  
    x = x + 1  
    print('g: x =', x)  
    h()  
    return x
```

```
x = 3
```

```
z = g(x)
```



Opseg - primer

```
def g(x):  
    def h():  
        x = 'abc'  
    x = x + 1  
    print('g: x =', x)  
    h()  
    return x
```

```
x = 3
```

```
z = g(x)
```

